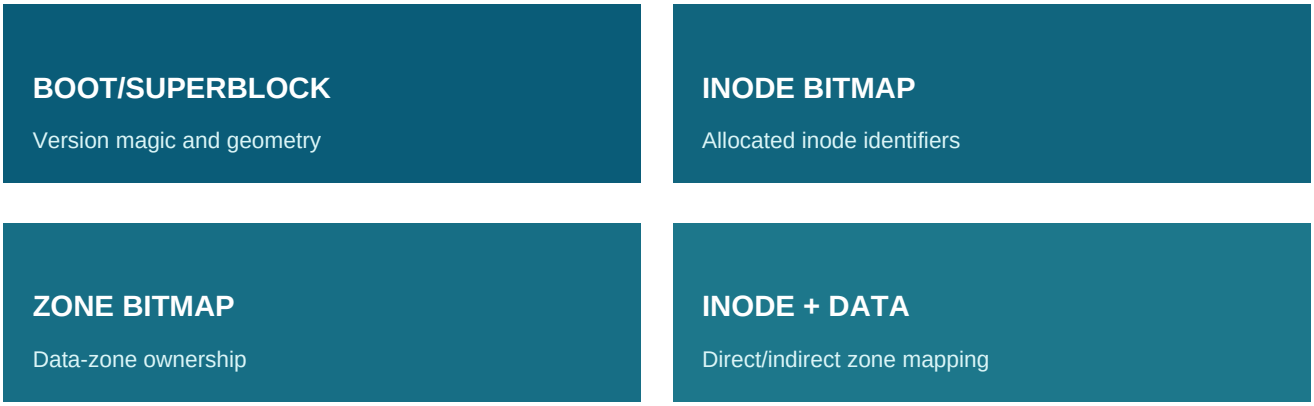
MINIX - teaching and Unix filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

A deliberately simple inode-and-zone disk format that evolved across MINIX V1, V2 and V3 for teaching and compact operating systems.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

MINIX - teaching and Unix filesystem

HISTORICAL CHRONOLOGY

Andrew S. Tanenbaum created MINIX in the 1980s as an educational Unix-like operating system for teaching operating-system design. Its filesystem was deliberately small and inspectable, with inodes and data zones rather than a complex journal or pooled allocator. The original design also influenced the early Linux period, when Linux initially supported MINIX media before ext-family filesystems superseded it.

MINIX disk formats evolved through V1, V2 and V3. The revisions expanded directory name lengths and representable zone addresses, while retaining a general Unix-like directory-to-inode model. A document that describes only one MINIX version is insufficient for identifying an arbitrary source image.

MINIX - teaching and Unix filesystem

ORIGINAL ON-DISK ARCHITECTURE

The volume contains boot area, superblock, inode bitmap, zone bitmap, inode table and data zones. The superblock identifies version-dependent magic, inode count, zone count, bitmap block counts, first data zone, log-zone size and size limits. MINIX V1 inodes use smaller zone addresses than later revisions, and V3 uses a distinct superblock representation.

Zones are the units through which file data is allocated, while block I/O can be a separate granularity. Direct pointers address initial zones, with single and double indirect mappings extending larger files. Linux kernel source demonstrates separate handling for V1, V2 and V3 geometry and guards against unsupported zone-size combinations.

MINIX - teaching and Unix filesystem

ALLOCATION AND DIRECTORIES

The inode bitmap identifies used inodes, and the zone bitmap tracks allocated data regions. Directories map names to inode numbers through fixed-layout version-specific entries. Original V1 name variants permitted short directory names, while later versions increased name capacity. Record width must be selected from the actual format variant; treating a V1 directory as V3 misaligns every following entry.

File size and zone pointers must be checked against declared geometry and file-block coverage. Hard links affect inode reference counts independently of data-zone ownership. Sparse-file or indirect-block behaviour, where permitted, must be validated with the exact version and system implementation.

MINIX - teaching and Unix filesystem

INTEGRITY AND RECOVERY

Traditional MINIX filesystems do not have a full metadata journal or native copy-on-write snapshots. Offline checks compare inode and zone bitmaps with the reachable directory graph and inode pointers. A crash may leave a bitmap entry set for an unreachable zone, or an inode referencing a zone not marked allocated. Both directions must be checked before any repair decision.

The superblock magic is only the first stage of format validation. A safe reader then verifies bitmap capacity, first data zone, inode bounds and indirect-pointer ranges, and refuses unsupported feature combinations. Guessing format based on the filename or copying only the superblock to repair an image is unsafe.

MINIX - teaching and Unix filesystem

DESIGN RATIONALE AND LIMITATIONS

MINIX optimised pedagogical clarity and compact code rather than massive filesystem size or high-concurrency workloads. Its simple separation between inode and zone bitmaps makes allocation easy to explain and implement. This is historically important in the development of educational and early Linux systems.

The price was restricted name lengths, file sizes and lack of journalling. Version-specific constraints matter more than a single aggregate MINIX capacity figure. Newer implementations sometimes interpret the same concepts with more robust validation, but that should not be projected back onto original MINIX V1 media.

MINIX - teaching and Unix filesystem

INFILTRATOROS INTEGRATION BOUNDARY

MINIX.so is an independent portable implementation with a restricted supported operation set. Its source and tests must separately demonstrate V1/V2/V3 identification and writing or recovery behaviour. A read-only module does not become a production writable driver simply because the historical filesystem supports writing.

The MINIX teaching-system source and Linux kernel’s parser are the appropriate references for version magic, geometry and inode addressing. The optional Linux kernel adapter requires separate boot/mount and failure-recovery qualification.

MINIX - teaching and Unix filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. MINIX 3 original source-code appendix and filesystem constants

https://www.minix3.org/documentation/AppendixB.pdf

2. Linux kernel MINIX filesystem source and geometry validation

https://code.googlesource.com/linux/torvalds/linux/+/7acf90feab8b009fde7def08ff2c622d0f10e99f/fs/minix/inode.c

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

Minix / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the Minix superblock and distinguish V1, V2 and V3 by the recorded magic and supported block-size information.

2. Resolve inode and zone bitmap placement from the superblock. Inode allocation and data-zone allocation use separate bitmaps.

3. Locate the inode table after the bitmap regions using the filesystem version’s inode size and structural definitions.

4. Minix V1 inodes use smaller zone-address fields than V2 and V3. The latter support wider zone pointers and different indirect-addressing capacity.

5. Follow seven direct zone references and the appropriate indirect levels for that version. A zone may consist of more than one physical block in some historical layouts.

6. Decode fixed-format directory entries according to the version’s name-length and record-layout rules rather than presuming ext-style variable records.

7. Locate inode one as the root directory in the conventional Minix on-disk format. Verify its directory type and zone mappings before traversal.

8. Keep the educational V1 design, later V2 expansion and Minix 3-era changes separate when describing historical limits.

Minix / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Validate that inode and zone bitmap sizes cover the declared objects; a superblock whose maps are too small cannot be safely interpreted.

2. Check first-data-zone, zone size, block size and total-zone counts together; arithmetic errors can place data reads inside the inode table.

3. Verify all direct and indirect zone references against valid data zones and detect loops or multiply owned allocation units.

4. Do not cast a V1 inode to a V2/V3 inode structure; differing field sizes can corrupt timestamps, ownership and data pointers.

5. Handle directory-entry name fields as bounded on-media byte arrays with version-specific lengths and termination rules.

6. Cross-check inode bitmap state, link counts and reachable directory records before reclaiming apparent orphaned inodes.

7. Distinguish Minix disk format metadata from the capabilities of a contemporary Linux driver, which may not support every historical zone-size combination.

8. Preserve the original superblock and bitmap regions when diagnosing corruption; rebuilding maps without traversing the namespace may destroy evidence.